

DOCUMENTACIÓN TÉCNICA

MANUAL DEL PROGRAMADOR

Sistema ERP Kamary — Arquitectura real del repositorio, contratos de código, servicios de dominio, integraciones, despliegue y procedimiento para extenderlo.

STACK

Laravel 10 · Inertia · React

VERSIÓN

2.0 · jul 2026

DIRIGIDO A

Equipo de desarrollo

Un solo contrato

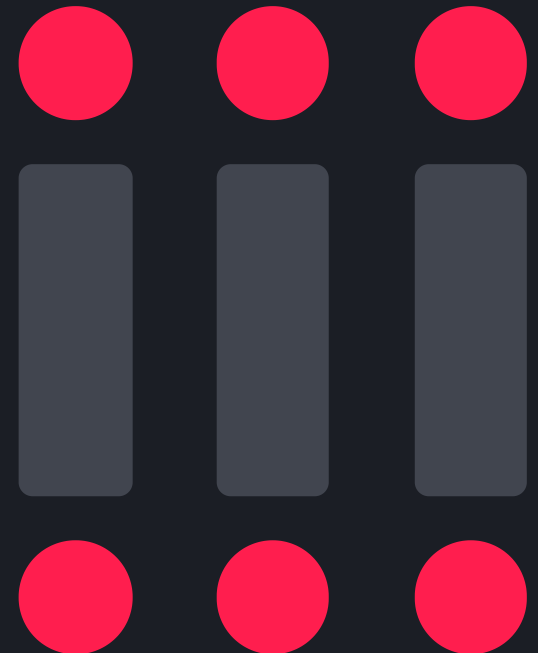
124 controladores comparten el mismo CRUD heredado. Conocer **BasicController** es conocer el 80 % del backend.

Permisos centralizados

Un catálogo único gobierna menú, rutas web, API y ámbito de empresa.

Recetas verificables

Cada procedimiento incluye los archivos exactos que hay que tocar y en qué orden.



CONTENIDO

ÍNDICE TÉCNICO

Este manual documenta el sistema tal como está construido. No propone una arquitectura ideal: describe los patrones reales del repositorio, dónde vive la lógica de negocio y cómo extenderla sin romper lo que ya está en producción.

01	Arquitectura general	03	14	Permisos: ModulePermissions	16
02	Ciclo de vida de un request	04	15	Middleware y seguridad	17
03	Estructura del repositorio	05	16	Servicios de dominio	18
04	Entorno de desarrollo local	06	17	Stock y kardex	19
05	Variables de entorno	07	18	Facturación: flujo de emisión	20
06	BasicController · propiedades	08	19	Facturación: configuración	21
07	BasicController · métodos	09	20	Guías de remisión (GRE 2.0)	22
08	BasicController · hooks y ejemplo	10	21	API externa de almacenamiento	23
09	Paginación, filtros y endpoints	11	22	Integraciones e-commerce	24
10	Frontend: anatomía de una pantalla	12	23	Receta: módulo nuevo (1/2)	25
11	Frontend: tabla, formularios y REST	13	24	Receta: módulo nuevo (2/2)	26
12	Modelo de datos por dominio	14	25	Pruebas, comandos y seeders	27
13	Ámbitos y reutilización	15	26	Despliegue y convenciones	28

DIMENSIÓN DEL CÓDIGO BASE

148

modelos Eloquent en `app/Models`

124

controladores bajo `Controllers/Admin`

88

pantallas React en `resources/js/Admin`

191

migraciones en `database/migrations`

01

SECCIÓN 01

ARQUITECTURA GENERAL

STACK

PIEZA	VERSIÓN / ROL
PHP	8.1+
Laravel	10.10
Inertia Laravel	1.2 — puente servidor/cliente
React	18.3 con Vite 5
Tailwind	3.4, conviviendo con el tema Adminto
MySQL	8.0
spatie/laravel-permission	6.7 — roles y permisos
Sanctum	3.3 — tokens de API
Intervention Image	2.7 — procesamiento de imágenes
sode/extend · sode-extend-react	utilidades propias de respuesta y fetch

CAPAS

- **Rutas** — `web.php` entrega pantallas; `api.php` entrega datos y ejecuta operaciones.
- **Controladores** — heredan de `BasicController`; declaran modelo, vista y hooks.
- **Servicios** — `app/Services`: stock, facturación, despacho, precios, integraciones.
- **Soporte** — `app/Support`: clases estáticas sin estado con reglas transversales.
- **Modelos** — Eloquent con `$fillable`, relaciones y casts.
- **Frontend** — una pantalla React por módulo, montada sobre el layout `Base`.

PRINCIPIOS VIGENTES

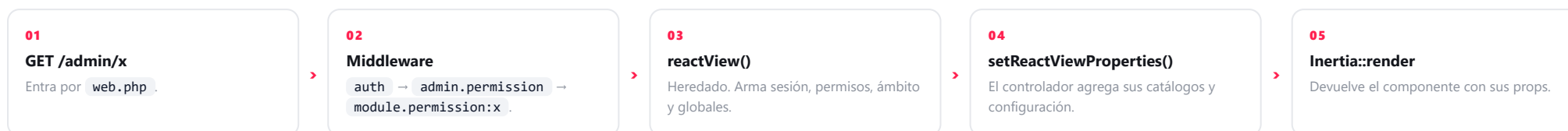
- Una pantalla = una ruta web + un controlador + un componente.
- Todo listado usa el mismo contrato de paginación: no se inventan formatos por módulo.
- El acceso se decide por permiso de módulo, nunca con comprobaciones ad hoc en la vista.
- La lógica que toca stock o comprobantes vive en servicios.
- El stock se deriva de documentos aprobados; no existe un saldo editable.
- Las bajas son lógicas: los documentos históricos no se eliminan.

Antes de introducir una librería o un patrón nuevo, verifique si el repositorio ya resuelve ese problema. La consistencia entre 88 pantallas es el activo más caro de este código: una pantalla que se comporta distinto obliga a documentar excepciones y multiplica el costo de mantenimiento.

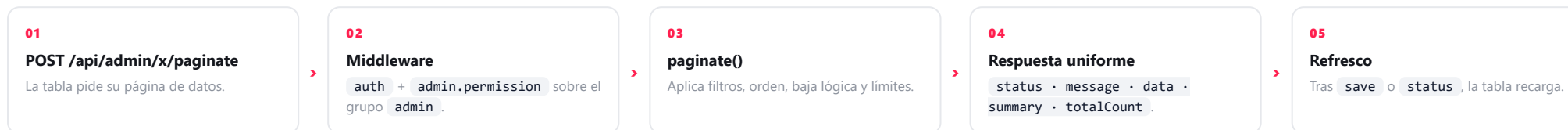
02 SECCIÓN 02

CICLO DE VIDA DE UN REQUEST

A · Carga de la pantalla · navegación del usuario



B · Operación sobre datos · acciones dentro de la pantalla



Props que toda pantalla recibe siempre

PROP	CONTENIDO
session	Usuario autenticado con sus permisos ya resueltos.
businessScopeKey	Empresa deducida de la ruta actual.
businessScopes	Etiquetas de las empresas disponibles.
global	APP_URL, dominio, claves públicas, tipo de cambio USD y EUR.

Regla estructural que no se negocia: `web.php` nunca devuelve datos y `api.php` nunca devuelve vistas. Lo que la pantalla necesita al abrirse viaja como prop desde `setReactViewProperties()`; todo lo demás se pide por API. Romper esta separación es lo que convierte un módulo en un caso especial imposible de mantener.

03 SECCIÓN 03

ESTRUCTURA DEL REPOSITORIO

```
kamary.pe/
├─ app/
│  ├─ Console/Commands/  // comandos artisan propios
│  ├─ Http/
│  │  ├─ Controllers/    // BasicController + Admin/**
│  │  │  ├─ Admin/       // 124 controladores del panel
│  │  │  │  ├─ Admin/Magistrales/
│  │  │  │  └─ Admin/Storage/
│  │  └─ Middleware/     // permisos, Inertia, tokens
│  ├─ Jobs/              // correo y WhatsApp diferidos
│  ├─ Models/            // 148 modelos
│  ├─ Observers/         // eventos de venta y usuario
│  ├─ Services/          // lógica de negocio
│  │  └─ Integrations/   // Ecomsur, Multivende
│  └─ Support/           // ámbitos, permisos, helpers
├─ database/
│  ├─ migrations/        // 191 archivos
│  └─ seeders/           // permisos, catálogos, demos
├─ resources/js/
│  ├─ Admin/             // 88 pantallas (+ Magistrales/)
│  ├─ Components/Adminto/ // layout, menú, VdTable, forms
│  ├─ Actions/           // BasicRest + *Rest por módulo
│  ├─ Reutilizables/     // filtros, rangos de fecha
│  └─ Utils/             // formato, export, sesión
├─ routes/               web.php · api.php · free.php
├─ facturador/           // app independiente (SUNAT)
├─ docs/                 // notas funcionales
└─ tests/                // PHPUnit Feature / Unit
```

Dónde va cada cosa

NECESITO...	UBICACIÓN CORRECTA
Pantalla nueva del panel	resources/js/Admin/X.jsx + Controllers/Admin/XController.php
Regla de stock	Services/StockService.php
Cálculo de comprobante	Services/BillingDocumentService.php
Llamada a un sistema externo	Services/Integrations/
Regla usada por varios módulos	app/Support (clase estática sin estado)
Endpoint del panel	api.php , grupo admin
Endpoint para terceros	api.php , grupo external o integrations
Proceso pesado o diferido	app/Jobs
Tarea manual de mantenimiento	app/Console/Commands

Carpeta facturador/ : aplicación independiente que se despliega como servicio aparte. No se modifica desde el ERP; la comunicación es siempre HTTP a través de `FacturadorPro5Service`.

04 SECCIÓN 04 ENTORNO DE DESARROLLO LOCAL

Instalación desde cero

```
# requisitos: PHP 8.1+, Composer, Node 18+, MySQL 8 (XAMPP)
git clone <repo> kamary.pe
cd kamary.pe

composer install
npm install

cp .env.example .env
php artisan key:generate

# crear la base kamary_db y configurar DB_* en .env
php artisan migrate
php artisan db:seed

# servir
php artisan serve --host=127.0.0.1 --port=8001

# frontend
npm run dev      # desarrollo con recarga
npm run build    # build que se despliega
```

El panel se sirve desde el build de Vite. `public/build` está versionado: si trabaja con `npm run dev`, ejecute `npm run build` antes de commitear o el servidor quedará con la versión anterior del frontend.

Datos iniciales

COMANDO	EFFECTO
<code>db:seed --class=ModulePermissionsSeeder</code>	Sincroniza el catálogo de permisos y los asigna a Admin y Root.
<code>db:seed --class=RoleSeeder</code>	Crea los roles base.
<code>db:seed --class=UsersSeeder</code>	Crea usuarios de arranque.
<code>SeedOperationalFlowDemoCommand</code>	Carga un escenario operativo completo para probar flujos.
<code>SeedStorageServiceDemoCommand</code>	Carga datos demo del servicio de almacenamiento.

Recomendaciones de trabajo

- Desarrolle con un usuario de rol **Admin**: evita perseguir permisos mientras programa.
- Pruebe además con un rol restringido antes de dar por terminado un módulo.
- Use `.env.testing` para la suite de pruebas; no corra tests contra su base de trabajo.
- Los seeders de demo **no** deben ejecutarse en producción.

05 SECCIÓN 05

VARIABLES DE ENTORNO

VARIABLE	PARA QUÉ SIRVE	OBSERVACIONES
APP_URL · APP_DOMAIN · APP_PROTOCOL	Construcción de enlaces absolutos y verificación pública de comprobantes.	Se inyectan como props globales a todas las pantallas.
APP_CORRELATIVE · APP_NAME	Identificación de la instalación.	Visibles en el frontend.
DB_*	Conexión MySQL.	En local <code>kamary_db</code> ; en Docker el host es el servicio <code>db</code> .
RSA_PUBLIC_KEY · RSA_PRIVATE_KEY	Cifrado asimétrico de credenciales enviadas desde el panel.	La pública viaja al frontend; la privada nunca sale del servidor.
FACTURADORPRO5_MODE	<code>api</code> = emisión real. Otro valor habilita el modo de plantillas demo.	Nunca dejar <code>api</code> apuntando a producción en un entorno de pruebas.
FACTURADORPRO5_BASE_URL	Host del facturador.	En Docker se resuelve por nombre de servicio dentro de <code>billing-network</code> .
FACTURADORPRO5_AUTH_MODE · API_EMAIL · API_PASSWORD	Autenticación contra el facturador.	Valores por defecto; cada empresa puede definir los suyos.
FACTURADORPRO5_*_ENDPOINT	Rutas de emisión, baja, estado, empresa, sedes y series.	Configurables para adaptarse a versiones del facturador.
FACTURADORPRO5_SERIES_*	Series por defecto de factura, boleta y nota de crédito.	La configuración por sede tiene prioridad.
FACTURADORPRO5_TIMEOUT · VERIFY_SSL	Tiempo máximo de espera y verificación de certificado.	SSL puede desactivarse solo en redes internas.
GMAPS_API_KEY	Mapas de zonas y puntos de entrega.	Puede sobrescribirse desde Datos generales en el panel.
MAIL_*	Envío de comprobantes y notificaciones por correo.	Usado por los Jobs de envío.
EVOAPI_* · WA_URL	Envío de mensajes por WhatsApp.	Opcional; si falta, el módulo queda inactivo.
CULQI_*	Pasarela de pagos.	Solo para los flujos de cobro en línea.

Nunca versione un `.env` real ni comparta claves por chat. Las credenciales del facturador y el certificado digital permiten emitir comprobantes fiscales a nombre de la empresa: su filtración es un incidente tributario, no solo técnico.

06

SECCIÓN 06

BASICCONTROLLER · PROPIEDADES DE CONFIGURACIÓN

Un controlador del panel casi no tiene código: declara propiedades y, cuando hace falta, sobrescribe hooks. Estas son las propiedades disponibles.

PROPIEDAD	POR DEFECTO	EFEECTO
<code>\$model</code>	<code>Model::class</code>	Modelo Eloquent sobre el que operan todos los métodos heredados.
<code>\$reactView</code>	<code>'Home'</code>	Componente React que se renderiza con Inertia. Ej: <code>'Admin/Articles'</code> .
<code>\$reactRootView</code>	<code>'admin'</code>	Plantilla Blade contenedora.
<code>\$imageFields</code>	<code>[]</code>	Campos de archivo. Se guardan en <code>storage/app/images/{tabla}</code> con nombre UUID y se sirven por <code>media()</code> .
<code>\$useIntervention</code>	<code>true</code>	Redimensiona imágenes a 1000 px de ancho máximo conservando proporción.
<code>\$with4get</code>	<code>[]</code>	Relaciones precargadas al pedir un registro puntual con <code>get()</code> .
<code>\$prefix4filter</code>	<code>null</code>	Prefijo de tabla para desambiguar columnas cuando el listado hace <i>joins</i> .
<code>\$ignorePrefix</code>	<code>[]</code>	Columnas que no deben recibir ese prefijo.
<code>\$identifier</code>	<code>'id'</code>	Clave usada para el <i>upsert</i> y para las operaciones por identificador.
<code>\$softDeletion</code>	<code>true</code>	Con <code>true</code> , <code>delete()</code> marca <code>status = null</code> en lugar de borrar la fila.
<code>\$booleanFields</code>	<code>[]</code>	Lista blanca de campos que pueden cambiarse con <code>boolean()</code> .
<code>\$ignoreStatusFilter</code>	<code>false</code>	Con <code>true</code> , el listado también devuelve los registros dados de baja.
<code>\$throwMediaError</code>	<code>false</code>	Con <code>false</code> , si la imagen no existe se devuelve una portada por defecto en vez de un error.

Por qué importa `$softDeletion` : el listado oculta automáticamente los registros con `status` nulo, de modo que una "eliminación" desaparece de la interfaz pero conserva la integridad referencial de los documentos históricos. Ponerlo en `false` solo tiene sentido en tablas auxiliares sin relaciones.

07 SECCIÓN 07

BASICCONTROLLER · MÉTODOS HEREDADOS

MÉTODO	RUTA TÍPICA	QUÉ HACE EXACTAMENTE
<code>reactView()</code>	GET /admin/x	Carga el usuario con sus permisos, el ámbito de la ruta, el tipo de cambio USD/EUR y las variables globales; fusiona lo que devuelva <code>setReactViewProperties()</code> y renderiza el componente con Inertia.
<code>paginate()</code>	POST /admin/x/paginate	Parte de <code>setPaginationInstance()</code> , aplica agrupación, filtro de baja lógica, filtros de columna, orden, conteo total y límites. Devuelve la respuesta uniforme.
<code>save()</code>	POST /admin/x	Abre transacción, ejecuta <code>beforeSave()</code> , procesa archivos declarados en <code>\$imageFields</code> , hace <i>upsert</i> por <code>\$identifier</code> , ejecuta <code>afterSave()</code> y confirma. Ante cualquier excepción revierte todo.
<code>status()</code>	PATCH /admin/x/status	Alterna el campo <code>status</code> entre 0 y 1 para el registro indicado.
<code>boolean()</code>	PATCH /admin/x/{field}	Actualiza un campo booleano puntual. Rechaza cualquier campo que no esté en la lista blanca.
<code>delete()</code>	DELETE /admin/x/{id}	Con <code>\$softDeletion</code> activo marca <code>status = null</code> ; en caso contrario elimina la fila. Si no afecta ningún registro, responde error.
<code>get()</code>	GET /admin/x/{id}	Devuelve un registro con las relaciones de <code>\$with4get</code> .
<code>media()</code>	GET /admin/x/media/{uuid}	Sirve el archivo asociado. Si no existe, devuelve una imagen por defecto en lugar de fallar.

Formato de respuesta

```
{
  "status": 200,
  "message": "Operación correcta",
  "data": [ ... ],
  "summary": [ ... ],
  "totalCount": 1240
}
```

Errores: los métodos capturan la excepción y responden `400` con el mensaje en `message`; el frontend lo muestra directamente al usuario. Por eso los mensajes de excepción deben estar escritos en español y ser comprensibles para quien opera el sistema, no solo para quien lo programa.

08

SECCIÓN 08

BASICCONTROLLER · HOOKS Y EJEMPLO COMPLETO

```
class ArticleController extends BasicController
{
    public $model      = Article::class;
    public $reactView   = 'Admin/Articles';
    public $imageFields = ['image'];
    public $booleanFields = ['visible', 'featured'];

    // 1. consulta base del listado
    public function setPaginationInstance(string $model)
    {
        return $model::with(['unit', 'laboratory'])
            ->where('module_scope', 'commercial');
    }

    // 2. totales al pie del listado (opcional)
    public function setPaginationSummary(string $model)
    {
        return [['column' => 'stock', 'summaryType' => 'sum']];
    }

    // 3. props iniciales de la pantalla
    public function setReactViewProperties(Request $request)
    {
        return [
            'units'      => Unit::where('status', true)->get(),
            'laboratories' => Laboratory::where('status', true)->get(),
        ];
    }

    // 4. normalización antes de persistir
    public function beforeSave(Request $request)
    {
        $body = $request->all();
        $body['code'] = strtoupper(trim($body['code'] ?? ''));
        return $body;
    }

    // 5. efectos posteriores, dentro de la MISMA transacción
    public function afterSave(Request $req, $jpa, bool $isNew)
    {
        if ($isNew) ArticlePresentation::createDefaults($jpa);
        return null;
    }
}
```

Cuándo usar cada hook

HOOK	USO CORRECTO
setPaginationInstance	Relaciones a precargar, <i>joins</i> , filtros fijos del módulo (por ejemplo, el <code>module_scope</code>).
setPaginationSummary	Totales o promedios al pie de la tabla.
setReactViewProperties	Catálogos y configuración que la pantalla necesita al abrirse. Evita una cascada de llamadas al montar.
beforeSave	Normalizar, completar campos derivados, validar reglas de negocio simples.
afterSave	Guardar el detalle del documento, mover stock, disparar eventos. Corre dentro de la transacción.

Si `afterSave()` lanza una excepción, se revierte también la cabecera. Esa es exactamente la conducta deseada en documentos con detalle: nunca debe quedar una nota guardada a medias. Aprovechélo en lugar de intentar controlar el error usted mismo.

Validaciones: las reglas simples se resuelven en `beforeSave()` lanzando una excepción con mensaje claro; ese texto llega tal cual al usuario.

09 SECCIÓN 09

PAGINACIÓN, FILTROS Y ENDPOINTS ESTÁNDAR

Request de paginación

```
{
  "filter": [{"name": "contains", "para": "and",
    ["status", "=", 1]],
  "sort": [{ "selector": "created_at", "desc": true }],
  "group": [{ "selector": "laboratory_id" }],
  "skip": 0,
  "take": 10,
  "requireTotalCount": true,
  "isLoadingAll": false
}
```

PARÁMETRO	COMPORTAMIENTO
filter	Gramática DevExtreme; la traduce <code>dxDataGrid::filter()</code> a cláusulas Eloquent.
sort	Si falta, ordena por <code>id</code> descendente.
group	Agrupar y devuelve claves distintas en lugar de filas.
isLoadingAll	Ignora <code>skip</code> / <code>take</code> : úselo solo para catálogos acotados.
requireTotalCount	Ejecuta un conteo adicional; desactívalo si no se muestra el total.

Endpoints por módulo

VERBO	RUTA	ACCIÓN
POST	/api/admin/x/paginate	Listado.
POST	/api/admin/x	Crear o actualizar.
PATCH	/api/admin/x/status	Baja lógica.
PATCH	/api/admin/x/{field}	Campo booleano permitido.
DELETE	/api/admin/x/{id}	Eliminar.
POST	/api/admin/x/import	Carga masiva (catálogos).

Con joins, declare `$prefix4filter` . Sin él, un filtro sobre una columna que existe en dos tablas produce un error SQL de columna ambigua que solo aparece cuando el usuario filtra, no en su prueba inicial.

Compatibilidad: el frontend migró de `dxDataGrid` a `VdTable` , pero se conservó la misma gramática de filtros. El backend no cambió con esa migración y no debe cambiar.

10 SECCIÓN 10

FRONTEND: ANATOMÍA DE UNA PANTALLA

```
import { createRoot } from 'react-dom/client'
import Base from '@Adminto/Base'
import VdTable from '@Adminto/VdTable'
import Modal from '@Adminto/Modal'
import InputFormGroup from '@Adminto/Form/InputFormGroup'
import CreateReactScript from '@Utils/CreateReactScript'
import ArticlesRest from '@Rest/Admin/ArticlesRest'

const articlesRest = new ArticlesRest()

const Articles = ({ units, laboratories }) => {
  const gridRef = useRef()
  const [isEditing, setIsEditing] = useState(false)

  const onSubmit = async (e) => {
    e.preventDefault()
    const result = await articlesRest.save(form)
    if (!result) return // el error ya se notificó
    setIsEditing(false)
    gridRef.current.refresh() // recarga el listado
  }

  return <>
    <VdTable ref={gridRef} rest={articlesRest} columns={columns} />
    <Modal title="Artículo" onSubmit={onSubmit}>
      <InputFormGroup label="Descripción" required />
    </Modal>
  </>
}

CreateReactScript((el, properties) => {
  createRoot(el).render(
    <Base {...properties} title="Artículos">
      <Articles {...properties} />
    </Base>
  )
})
```

Qué aporta cada pieza

PIEZA	RESPONSABILIDAD
CreateReactScript	Punto de montaje: recibe las props que envió el controlador y monta el árbol.
Base	Layout completo: menú lateral, barra superior, notificaciones y modales globales.
Menu	Construye el menú evaluando <code>canAccess(permiso)</code> ítem por ítem.
VdTable	Listado con filtros por columna, orden, paginación y exportación.
Modal	Formulario en ventana modal, patrón único para altas y ediciones.
*FormGroup	Campos ya estilizados y validados visualmente.
*Rest	Cliente HTTP del módulo: encapsula rutas, XSRF y mensajes de error.

El panel no incluye FontAwesome. Los iconos disponibles son los del tema (`ti ti-*` , `ri-*`) y, en algunos módulos, `mdi mdi-*` . Usar clases `fa-*` produce iconos invisibles: el elemento existe pero no se ve nada.

11 SECCIÓN 11

TABLA, FORMULARIOS Y CLIENTES REST

BasicRest: el cliente heredado

```
class ArticlesRest extends BasicRest {  
  path = isMagistralesPath()  
    ? 'admin/magistrales/articles'  
    : (isStoragePath() ? 'admin/storage/articles'  
                        : 'admin/articles')  
  
  // métodos propios del módulo  
  getUnits = async () => { ... }  
  importRows = async (rows) => { ... }  
}
```

MÉTODO HEREDADO	ENDPOINT QUE CONSUME
<code>paginate(params)</code>	POST /api/{path}/paginate
<code>save(request)</code>	POST /api/{path}
<code>status({id, status})</code>	PATCH /api/{path}/status
<code>boolean({id, field, value})</code>	PATCH /api/{path}/{field}
<code>delete(id)</code>	DELETE /api/{path}/{id}
<code>get(id)</code>	GET /api/{path}/{id}
<code>simpleGet / simplePost</code>	Atajos para endpoints puntuales del módulo.

Comportamiento incluido

- Adjunta el token **XSRF** en cada llamada.
- Muestra el error con `toast` y devuelve `null`: la pantalla solo comprueba el resultado.
- Traduce respuestas HTTP conocidas: **413** archivo demasiado grande, **419** sesión expirada, **500** error interno.
- Soporta envío con archivos mediante `hasFiles`.

Campos de formulario disponibles

`InputFormGroup`, `SelectFormGroup`, `SelectAPIFormGroup`, `SwitchFormGroup`, `CheckboxFormGroup`, `RadioFormGroup`, `TextareaFormGroup`, `ImageFormGroup`, `QuillFormGroup`, `EditorFormGroup`, `PasswordFormGroup`, `UbigeoCascade`.

Ámbito por ruta: `permissionScope` expone `isMagistralesPath()` e `isStoragePath()`. Gracias a eso, una misma pantalla React sirve a varias líneas de negocio apuntando a distintos endpoints, sin duplicar componentes.

No llame a `fetch` directamente desde una pantalla. Perdería XSRF, el manejo de errores y la uniformidad de mensajes. Si necesita un endpoint nuevo, agréguelo como método del `*Rest` del módulo.

12 SECCIÓN 12

MODELO DE DATOS POR DOMINIO

DOMINIO	MODELOS PRINCIPALES	NOTAS DE IMPLEMENTACIÓN
Catálogos	Article , ArticlePresentation , Unit , Laboratory , ActivePrinciple , Supplier , Warehouse , WarehouseLocation , Batch	El código de artículo y el símbolo de unidad son únicos a nivel global, no por línea de negocio.
Inventario	EntryNote + EntryNoteItem , ExitNote + ExitNoteItem , InventoryCount	El saldo se deriva de documentos aprobados; no hay columna de stock editable.
Compras	PurchaseOrder , PurchaseReceipt , AccountsPayable + cuotas y pagos	La recepción genera lotes vía <code>StockService::syncPurchaseReceiptBatches()</code> .
Comercial	Client , ClientDeliveryAddress , PriceList + ítems, CommercialOrder + ítems, TakeOrder , AccountsReceivable	El pedido tiene cuatro estados independientes y su propia bitácora de eventos.
Despacho	Dispatch , DispatchAssignment , Driver , Vehicle , Zone , DeliveryEvidence , DeliveryDelayReason	Agrupar pedidos preparados y registra evidencia y motivos de demora.
Facturación	BillingDocument + ítems, BillingEvent , ReferralGuide + ítems, Serie , Business , BusinessBranch	BillingEvent guarda la traza completa de cada intercambio con el facturador.
Magistrales	MagistralFormula + ítems e historial, MagistralProductionOrder , MagistralSale , MagistralResponsible , MagistralFormat	Reutiliza catálogos de almacén; se separa por <code>module_scope</code> .
Almacenamiento	StorageProductLot , StorageLocation , ClientContract , ClientStorageTariff , ClientNotification , StorageApiToken	Mercadería de terceros: el stock se atribuye al cliente depositante.
Integraciones	IntegrationMapping , IntegrationLog , CommercialOrderTrackingEvent	Correspondencia de identificadores externos y traza de cada intercambio.

Al escribir migraciones, respete las claves únicas existentes y recuerde que varios catálogos son compartidos entre líneas de negocio. Una unicidad mal definida no falla en desarrollo: falla al ejecutar una importación masiva en producción, cuando ya hay decenas de miles de filas.

13 SECCIÓN 13

ÁMBITOS Y REUTILIZACIÓN DE CONTROLADORES

El sistema atiende a dos empresas y a varias líneas de negocio con las mismas tablas y, en buena parte, las mismas pantallas. Tres mecanismos hacen posible esa reutilización sin mezclar datos.

BusinessScope

- Deduce de la ruta actual a qué empresa pertenece la pantalla.
- Se envía al frontend como `businessScopeKey`.
- Alimenta la separación visual del menú entre Kamary Perú y Kamary Medical.

StorageScope

- Restringe las consultas al inventario del servicio de almacenamiento.
- Evita que mercadería de terceros aparezca en pantallas comerciales.

module_scope

- Columna que separa catálogos compartidos por línea de negocio: comercial, magistrales, muestras, almacenamiento.
- Permite reutilizar tablas sin duplicar módulos.

Herencia de controladores

```
// Admin/Storage/ClientController.php
class ClientController extends BaseClientController
{
    protected function clientModuleScope(): ?string
    {
        return 'storage';
    }

    public function setReactViewProperties(Request $request)
    {
        return array_merge(parent::setReactViewProperties($request), [
            'sectionTitle' => 'Clientes de almacenamiento',
            'requiredPermission' => 'storage-clients',
            'storageContext' => true,
        ]);
    }

    public function beforeSave(Request $request)
    {
        $request->merge([
            'module_scope' => 'storage',
            'has_storage_service' => true,
        ]);
        return parent::beforeSave($request);
    }
}
```

El patrón, en tres reglas

- 01 El controlador especializado **hereda** del genérico y solo declara su ámbito.
- 02 `setReactViewProperties()` agrega props de contexto: título, permiso requerido y banderas que la pantalla usa para adaptarse.
- 03 `beforeSave()` fuerza el `module_scope` para que un registro creado en una sección no pueda aparecer en otra.

Nunca confíe el ámbito al frontend. La pantalla puede enviar cualquier cosa: es el controlador el que impone `module_scope` en `beforeSave()` y el que filtra en `setPaginationInstance()`. Si el filtro solo existe en la interfaz, tarde o temprano un registro se cruza de línea de negocio.

14

SECCIÓN 14

PERMISOS: MODULEPERMISSIONS

Toda la autorización del panel se apoya en una sola clase: `app/Support/ModulePermissions.php`. Si un módulo no está declarado ahí, no existe para el sistema de permisos.

Métodos

MÉTODO	QUÉ RESUELVE
<code>permissions()</code>	Catálogo único: clave del permiso y su nombre legible. Toda alta de módulo empieza aquí.
<code>webModules()</code>	Mapea permiso → ruta del panel. Define además el orden para elegir la pantalla de inicio.
<code>userCan()</code>	Resuelve permiso + ámbito + rol superusuario. Es la función que se debe usar siempre.
<code>scopeForPermission()</code>	Los permisos que empiezan con <code>storage-</code> son de Kamary Medical; el resto, de Kamary Perú.
<code>userScopeAllows()</code>	Comprueba el arreglo JSON de <code>users.scope</code> .
<code>isSuperUser()</code>	Reconoce los roles Admin, Root y equivalentes.
<code>homePathForUser()</code>	Primer módulo accesible tras iniciar sesión.
<code>sync()</code>	Crea los permisos faltantes y los asigna a Admin y Root. Es idempotente.

Cómo se declara un permiso

```
// 1. catálogo
'x' => 'Área - Módulo X',

// 2. mapeo permiso -> ruta
['permission' => 'x',
 'web'        => '/admin/x',
 'label'      => 'Módulo X'],

// 3. sincronizar
php artisan db:seed --class=ModulePermissionsSeeder
```

Reglas de resolución

- 01 Si el usuario es superusuario (Admin / Root) → acceso total.
- 02 Si el ámbito del usuario no incluye la empresa del permiso → denegado, aunque el rol lo tenga.
- 03 Si el rol no tiene el permiso → denegado.
- 04 Un permiso inexistente en la base no lanza excepción: se resuelve como denegado.

El ámbito vive en `users.scope` como arreglo JSON con los valores `kamary-peru` y/o `kamary-medicals`. Es la causa más frecuente de "tengo el permiso pero no veo el módulo": el rol es correcto, el ámbito no.

15 SECCIÓN 15

MIDDLEWARE Y SEGURIDAD

MIDDLEWARE	RESPONSABILIDAD	DÓNDE SE APLICA
auth	Exige sesión válida.	Grupo raíz de <code>web.php</code> y del bloque autenticado de <code>api.php</code> .
admin.permission	Valida que el usuario pueda acceder a la ruta <code>/admin/*</code> solicitada, comparándola con el mapa de módulos.	Grupo <code>admin</code> en web y api.
module.permission:x	Exige un permiso concreto. Acepta varias claves como alternativas: basta con tener una.	Rutas individuales que necesitan una restricción adicional.
storage.api.token:hab	Autentica el token del cliente externo y verifica su habilidad (<code>stock:read</code> , <code>orders:write</code> , ...).	Grupo <code>external/storage</code> .
HandleInertiaRequests	Comparte sesión, permisos y datos globales con todas las vistas.	Grupo web.
VerifyCsrfToken	Protege las llamadas del panel; el cliente REST adjunta el token XSRF automáticamente.	Grupo web.
EnsureInternalRouteToken	Protege rutas internas que no deben exponerse al exterior.	Rutas puntuales.

Otros controles vigentes

- **Cifrado RSA:** las credenciales sensibles enviadas desde el panel viajan cifradas con la clave pública y se descifran en el servidor con la privada.
- **Tokens de API:** se guardan con prefijo visible (`kst_...`) y valor cifrado, con habilidades granulares y expiración opcional.
- **Baja lógica:** el histórico documental nunca se elimina físicamente.
- **Trazabilidad:** los documentos registran usuario y fecha de creación y modificación.

Toda ruta nueva del panel debe nacer protegida. Una ruta agregada al grupo `admin` sin su `module.permission` queda accesible para cualquier usuario autenticado que conozca la URL. Al crear un módulo, la ruta y el permiso se escriben en el mismo commit, nunca en dos etapas.

Al exponer un endpoint público (integraciones o API de clientes) verifique tres cosas: autenticación por token, verificación de habilidad e idempotencia. Sin la tercera, un reintento del cliente duplica documentos y movimientos de stock.

16 SECCIÓN 16

SERVICIOS DE DOMINIO

Antes de escribir lógica en un controlador, verifique si ya existe el servicio correspondiente. Estos son todos los que hay hoy en `app/Services`.

SERVICIO	RESPONSABILIDAD	A TENER EN CUENTA
StockService	Stock disponible y neto por almacén, lote y ubicación; subconsultas de kardex; sincronización de lotes en recepciones.	Toda validación de disponibilidad pasa por aquí. No consulte saldos con queries propias.
BillingDocumentService	Preparación, emisión, anulación y notas de crédito; sincronización de estado y registro de eventos.	Traduce el documento interno al payload del facturador y actualiza el pedido origen.
FacturadorPro5Service	Cliente HTTP del facturador: emisión, baja, consulta, descarga de PDF/XML/CDR y sincronización de empresa, sedes y series.	<code>forBusiness()</code> selecciona credenciales de la empresa emisora antes de cada llamada.
FacturadorPro5SettingsService	Resolución de configuración efectiva (entorno vs. empresa).	Punto único para saber con qué credenciales se emitirá.
BusinessFacturadorSyncService	Alta y sincronización de la empresa en su instancia de facturador.	Sube logo y certificado, replica establecimientos y series.
ReferralGuideService	Guías de remisión electrónicas (GRE 2.0) con emisión por ticket y consulta posterior.	La fecha de emisión se calcula en hora de Perú aunque el servidor corra en UTC.
CommercialOrderStockService	Reserva y liberación de stock según el ciclo del pedido.	Se dispara al cambiar estados; evita descuentos duplicados.
CommercialOrderTrackingService	Bitácora de eventos del pedido.	Fuente de la trazabilidad que ve el usuario en el detalle.
DispatchService	Armado de despachos, asignación de vehículo y conductor, cierre de entregas.	Coordina el paso de preparación a ruta.
PriceListResolverService	Precio aplicable a un cliente y artículo según lista vigente y prioridad.	Devuelve el precio base del artículo si no hay lista aplicable.
AccountsPayableService AccountsReceivableService	Generación de cuotas, aplicación de pagos y cálculo de saldos.	El saldo nunca se escribe a mano: se recalcula desde los movimientos.
BillingDocumentVerificationService	Verificación pública de comprobantes por enlace firmado.	Alimenta la página que consulta el cliente final.
Integrations/*	<code>EcomsurOmsService</code> , <code>MultivendeClient</code> , <code>ExternalOrderEventService</code> .	Todo diálogo con plataformas externas se encapsula aquí.

Regla: un controlador puede orquestar servicios, pero no contener reglas de negocio. Si necesita repetir un cálculo en dos módulos, corresponde extraerlo a `app/Services` (con dependencias) o a `app/Support` (sin estado).

17 SECCIÓN 17

STOCK Y KARDEX

No existe una columna con el stock. El saldo se calcula sumando entradas y restando salidas sobre documentos aprobados. Entender esto evita el error más caro que se puede cometer en este sistema.

Métodos de StockService

MÉTODO	DEVUELVE
<code>getAvailableStockByWarehouse()</code>	Disponible de un artículo en un almacén, con posibilidad de excluir un documento en edición.
<code>getNetStockByWarehouse()</code>	Saldo neto sin descontar compromisos.
<code>getAvailableStockByStorageKey()</code>	Disponible por la combinación completa: artículo, almacén, lote, vencimiento y ubicación.
<code>availableStorageStockRows()</code>	Filas de existencia para el servicio de almacenamiento.
<code>storageLocationOccupancyRows()</code>	Ocupación por ubicación física.
<code>incomingKardexMovementsQuery()</code>	Consulta base de movimientos para el kardex.
<code>syncPurchaseReceiptBatches()</code>	Crea o actualiza los lotes al confirmar una recepción de compra.

La clave de existencia

Una existencia no es "artículo + cantidad". Es la combinación de:

artículo + almacén + lote + vencimiento + ubicación

Nunca escriba una consulta propia de stock. Los cálculos consideran documentos aprobados, exclusiones del documento en edición y separación por lote y ubicación. Una consulta ad hoc "que parece dar lo mismo" produce diferencias que solo se detectan semanas después, cuando ya hay despachos hechos sobre stock inexistente.

Al editar un documento aprobado use el parámetro de exclusión que ofrecen los métodos: sin él, el propio documento se cuenta a sí mismo y el disponible aparece menor de lo real.

18 SECCIÓN 18

FACTURACIÓN ELECTRÓNICA: FLUJO DE EMISIÓN

> El ERP no firma comprobantes: los delega en una instancia de facturador por empresa

Red interna: `billing-network`

Emisión, paso a paso en el código

- 01 `prepareDocument()` — normaliza cliente, ítems, impuestos y totales.
- 02 `buildConnectorPayload()` — arma el cuerpo que espera el facturador.
- 03 `FacturadorPro5Service::forBusiness()` — resuelve URL y credenciales de la empresa emisora.
- 04 `issue()` — envía el documento y devuelve la respuesta del proveedor.
- 05 El resultado se guarda en el documento y se registra un `BillingEvent`.
- 06 `syncSourceBillingStatus()` — actualiza el estado de facturación del pedido origen.

Otras operaciones

OPERACIÓN	MÉTODO
Anulación (baja)	<code>cancelDocument()</code> → <code>cancel()</code>
Nota de crédito	<code>createCreditNote()</code> → <code>creditNote()</code>
Consulta de estado	<code>syncProviderStatus()</code> → <code>status()</code>
Descarga PDF/XML/CDR	<code>downloadFile()</code>

Puntos delicados ya resueltos

- Los importes de las **notas de crédito** se envían en positivo; el tipo de documento define el signo.
- Las **bajas** usan su propio endpoint y su propio flujo de estado, distinto al de emisión.
- La **fecha y hora de emisión** se calculan en zona horaria de Perú para evitar rechazos por desfase cuando el servidor corre en UTC.
- Cada intercambio queda en `BillingEvent`: es la evidencia ante una discrepancia con SUNAT.

Antes de tocar la construcción del payload, ejecute `FacturadorPro5ServiceTest`. Ese test existe precisamente porque los detalles de formato son el origen histórico de los rechazos, y una corrección "obvia" suele romper otro tipo de documento.

Verificación pública: cada documento expone un enlace firmado que permite al cliente final validar el comprobante y descargar PDF, XML y CDR.

19 SECCIÓN 19

FACTURACIÓN: CONFIGURACIÓN Y ENDPOINTS

Endpoints del facturador

FUNCIÓN	RUTA POR DEFECTO
Autenticación	/api/auth/login
Emisión y nota de crédito	/api/documents
Consulta de estado	/api/documents/status
Anulación (baja)	/api/voided
Estado de la baja	/api/voided/status
Empresa	/api/company
Establecimientos	/api/establishments
Series	/api/series

Todas las rutas son configurables por entorno, de modo que una actualización del facturador no obliga a tocar código.

Credenciales por empresa

- 01** Cada `Business` puede definir su propia URL, usuario y contraseña del facturador.
- 02** Si no las define, se usan los valores del entorno.
- 03** `forBusiness()` resuelve la configuración efectiva antes de cada llamada.
- 04** `BusinessFacturadorSyncService` sube logo y certificado y replica sedes y series.

Una instancia de facturador = una empresa. El facturador es monoempresa: por eso el `docker-compose` levanta instancias completas e independientes (app + nginx + base de datos) por cada razón social, y el ERP se conecta a cada una por su nombre de servicio dentro de la red de facturación.

Antes de la primera emisión

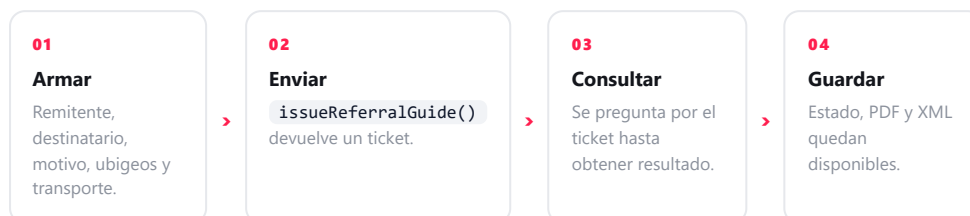
- Certificado digital vigente cargado con su clave.
- Logo de la empresa cargado.
- Credenciales del facturador verificadas.
- Series sincronizadas y coincidentes con las declaradas ante SUNAT.

20 SECCIÓN 20

GUÍAS DE REMISIÓN ELECTRÓNICAS (GRE 2.0)

La GRE no se emite igual que una factura: usa API REST con OAuth2 y devuelve un **ticket** que hay que consultar después. Ese asincronismo condiciona todo el diseño del módulo.

Flujo



MÉTODO	USO
<code>issueReferralGuide()</code>	Emisión de la guía.
<code>cancelReferralGuide()</code>	Anulación con motivo.
<code>downloadReferralGuideFile()</code>	Descarga de PDF o XML.
<code>buildReferralGuideCancelRequestBody()</code>	Construcción del cuerpo de la baja.

Detalles de implementación

- **Hora de Perú:** la fecha y hora de emisión se calculan en `America/Lima`. Con el servidor en UTC, usar la hora del sistema provoca el rechazo por fecha futura.
- **Emisión manual:** existe un módulo que emite la guía sin mover inventario, para traslados ya registrados por otra vía.
- **Datos del transporte:** placa y breveté se validan del lado de SUNAT; deben venir de los catálogos de Vehículo y Conductor.
- **Credenciales GRE por empresa:** se configuran junto con las del facturador.

El ticket no es la aceptación. Recibir ticket significa que SUNAT tomó el envío, no que la guía es válida. El módulo debe consultar el resultado y solo habilitar la descarga del PDF cuando el estado sea aceptado; entregar al conductor un PDF de una guía no aceptada es un problema en fiscalización.

21 SECCIÓN 21

API EXTERNA DE ALMACENAMIENTO

Permite que un cliente del servicio de almacenamiento consulte su stock y cree pedidos desde su propio ERP. Base:

`/api/external/storage` .

Endpoints y habilidades

VERBO	RUTA	HABILIDAD
GET	<code>/me</code>	token válido
GET	<code>/stock</code>	<code>stock:read</code>
POST	<code>/orders</code>	<code>orders:write</code>
GET	<code>/orders/{referencia}</code>	<code>orders:read</code>

```
Authorization: Bearer kst_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
# alternativa
X-Storage-Token: kst_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Filtros de `/stock`

`warehouse_id` , `q` / `search` , `sku` , `lot` , `location` , `page` , `per_page` (máximo 100).

Creación de pedidos

```
{
  "external_reference": "ERP-000123",
  "source": "erp_cliente",
  "warehouse_id": 2,
  "exit_date": "2026-06-18",
  "items": [{
    "sku": "SKU-001",
    "lot": "L-001",
    "expiration_date": "2027-12-31",
    "location": "A-01",
    "quantity": 3
  }]
}
```

- Un pedido aceptado crea una **nota de salida aprobada** y descuenta stock.
- `external_reference` es **idempotente** por cliente y origen: reenviar la misma referencia devuelve el pedido existente con `idempotent: true` .
- Cada ítem exige `sku` o `article_id` , y `lot` es obligatorio.
- Sin stock suficiente responde `422` y no crea nada.
- Errores: `401` token inválido, `403` sin habilidad o cliente no habilitado, `422` payload o existencias.

El token está asociado a un solo cliente. La API nunca devuelve ni descuenta stock de otro, y esa restricción se aplica en el servidor, no en el filtro que envíe el cliente. Se administran desde `/admin/storage-api-tokens` o con `php artisan storage-api:token` .

22 SECCIÓN 22

INTEGRACIONES DE CANALES DE VENTA

Endpoints públicos

RUTA	FUNCIÓN
POST /api/integrations/eomsur/logistic-orders	Recepción de órdenes logísticas del OMS.
POST /api/integrations/eomsur/stock	Publicación de disponibilidad hacia el canal.
POST /api/integrations/multivende/webhook	Webhook de pedidos del marketplace.

Piezas involucradas

CLASE	ROL
EomsurOmsService	Diálogo con la plataforma Eomsur.
MultivendeClient	Cliente del marketplace.
ExternalOrderEventService	Normaliza el evento externo y lo convierte en pedido interno.
IntegrationMapping	Correspondencia entre identificadores externos e internos.
IntegrationLog	Traza de cada intercambio, para diagnóstico.

Reglas de diseño para integraciones

- 01 Idempotencia obligatoria:** el mismo mensaje recibido dos veces no debe duplicar documentos ni movimientos.
- 02 Registrar siempre:** toda llamada entrante y saliente deja rastro en `IntegrationLog`.
- 03 Mapear, no adivinar:** las equivalencias de identificadores se guardan en `IntegrationMapping`, nunca se deducen por nombre.
- 04 Fallar de forma explícita:** un payload inválido responde error claro; no se acepta silenciosamente lo que no se entiende.
- 05 Trabajo igual al propio:** el pedido importado sigue el mismo ciclo de estados que uno cargado a mano.

Los reintentos son la norma, no la excepción. Las plataformas externas reenvían el mismo webhook ante cualquier duda de entrega. Un endpoint sin idempotencia genera pedidos duplicados y descuentos de stock fantasma que nadie relaciona con la integración hasta que el inventario ya no cuadra.

23

SECCIÓN 23

RECETA: CREAR UN MÓDULO NUEVO (1/2)

Backend completo. Los pasos están en el orden en que conviene ejecutarlos: cada uno se puede probar antes de seguir con el siguiente.

```
// PASO 1 · migración
php artisan make:migration create_xs_table

Schema::create('xs', function (Blueprint $t) {
    $t->id();
    $t->string('name');
    $t->string('code')->unique();
    $t->foreignId('business_id')->nullable();
    $t->boolean('status')->nullable()->default(true);
    $t->timestamps();
});

// PASO 2 · modelo
class X extends Model {
    protected $fillable = ['name','code','business_id','status'];
    protected $casts = ['status' => 'boolean'];

    public function business() {
        return $this->belongsTo(Business::class);
    }
}

// PASO 3 · controlador
class XController extends BasicController {
    public $model = X::class;
    public $reactView = 'Admin/Xs';

    public function setPaginationInstance(string $model) {
        return $model::with(['business']);
    }
}
```

```
// PASO 4a · routes/web.php (la pantalla)
Route::get('/x', [XController::class, 'reactView'])
    ->middleware('module.permission:x');

// PASO 4b · routes/api.php (las operaciones)
Route::post('/x', [XController::class, 'save']);
Route::post('/x/paginate', [XController::class, 'paginate']);
Route::patch('/x/status', [XController::class, 'status']);
Route::patch('/x/{field}', [XController::class, 'boolean']);
Route::delete('/x/{id}', [XController::class, 'delete']);

// PASO 5 · permiso (app/Support/ModulePermissions.php)
// en permissions()
'x' => 'Área - Módulo X',

// en webModules()
['permission' => 'x',
 'web' => '/admin/x',
 'label' => 'Módulo X'],

// y sincronizar
php artisan db:seed --class=ModulePermissionsSeeder
```

El orden de PATCH importa. `/x/status` debe declararse antes que `/x/{field}`: al revés, la ruta comodín captura la palabra `status` y el interruptor de baja lógica deja de funcionar.

24 SECCIÓN 24

RECETA: CREAR UN MÓDULO NUEVO (2/2)

Frontend, menú y verificación final.

```
// PASO 6 · cliente REST
// resources/js/Actions/Admin/XsRest.js
import BasicRest from '../BasicRest'

class XsRest extends BasicRest {
  path = 'admin/x'
}
export default XsRest

// PASO 7 · menú
// resources/js/Components/Admintto/Menu.jsx
{canAccess('x') &&
  <MenuItem href='/admin/x' icon='ti ti-box'>
    Módulo X
  </MenuItem>}

// PASO 8 · pantalla
// resources/js/Admin/Xs.jsx -> copie un módulo similar
// catálogo simple -> Units.jsx / Laboratories.jsx
// con detalle -> EntryNotes.jsx
// con documento fiscal -> BillingDocuments.jsx

// PASO 9 · compilar
npm run build
```

Verificación antes de dar por terminado

- 01** Permiso: el módulo aparece y desaparece según el rol, y la URL directa también queda bloqueada.
- 02** Ámbito: un usuario de la otra empresa no lo ve.
- 03** Listado: filtros por columna, orden, paginación y exportación funcionan.
- 04** Alta y edición: validaciones correctas y mensajes en español.
- 05** Baja lógica: el interruptor de estado oculta el registro sin borrarlo.
- 06** Relaciones: el registro se puede seleccionar desde los módulos que lo consumen.
- 07** Build: `npm run build` ejecutado e incluido en el commit.

Copie un módulo equivalente antes que empezar de cero. No es pereza: es lo que garantiza que las 88 pantallas se comporten igual. Un módulo con su propio criterio de filtros o de mensajes obliga a documentar una excepción y a explicarla a cada usuario nuevo.

Si el módulo pertenece a una línea de negocio (magistrales, almacenamiento, muestras), no cree tablas paralelas: use `module_scope` y herede del controlador genérico, como se explica en la sección 13.

25 SECCIÓN 25

PRUEBAS, COMANDOS Y CARGA DE DATOS

Suite de pruebas

PRUEBA	CUBRE
InventoryKardexTest	Saldos y movimientos del kardex.
EntryNotesTest · ExitNotesTest	Documentos de inventario y su efecto en stock.
CommercialOrdersTest	Ciclo del pedido y transición de estados.
FacturadorPro5ServiceTest	Payloads de emisión y anulación.
MagistralesStockFlowsTest	Consumo de insumos y producto terminado.
MagistralesModuleReadinessTest	Disponibilidad de los módulos magistrales.
StorageServiceKmFlowTest	Flujo completo del servicio de almacenamiento.
ArticleImportTypesTest · SupplierImportTest	Importaciones masivas desde Excel.
WarehouseBranchRelationTest · BusinessBranchesTest	Relación de almacenes y sedes con empresas.
DeliveryEvidenceTest	Evidencia de entrega.

```
php artisan test
php artisan test --filter=Kardex
```

Comandos propios

COMANDO	USO
storage-api:token	Genera un token de API para un cliente de almacenamiento.
IntegrationMapCommand	Mapea identificadores externos con registros internos.
AuditOperationalScopeCommand	Audita permisos y ámbitos configurados.
SeedOperationalFlowDemoCommand	Escenario operativo completo de prueba.
SeedStorageServiceDemoCommand	Datos demo del servicio de almacenamiento.

Seeders

- `ModulePermissionsSeeder` — sincroniza permisos; ejecutable tantas veces como haga falta.
- `RoleSeeder` · `UsersSeeder` — roles y usuarios base.
- `KamaryPeruImportSeeder` · `StorageServiceImportSeeder` — importaciones controladas por centinela: se ejecutan una sola vez.
- `Demo*` — nunca en producción.

Antes de tocar stock o facturación, ejecute la suite completa. Esos módulos tienen efectos en cadena (kardex, cuentas, comprobantes) y las pruebas existentes son la red que evita romper documentos ya emitidos.

26 SECCIÓN 26

DESPLIEGUE Y CONVENCIONES DE TRABAJO

Servicios del compose

SERVICIO	ROL
app	Aplicación Laravel (PHP-FPM), con volúmenes de <code>storage</code> y caché.
db	MySQL 8 con volumen persistente.
nginx	Servidor web del ERP.
facturador-app facturador-nginx facturador-db	Instancia de facturación de la primera empresa.
facturador2-*	Instancia independiente para la segunda empresa.

- Redes: `app-network-kamary`, `billing-network` y una red propia por facturador.
- Certificados, comprobantes firmados, PDF y CDR se montan en volúmenes: sobreviven a la recreación del contenedor.
- El facturador se autoconfigura al arrancar (migraciones y seeders controlados por variables).

Puesta en producción

```
# 1. compilar ANTES de subir
npm run build

# 2. commit incluyendo public/build
git add -A
git commit -m "mensaje corto en español"
git push origin main

# 3. en el servidor
git pull
docker compose build app
docker compose up -d

# 4. migraciones dentro del contenedor
docker compose exec app php artisan migrate --force
docker compose exec app php artisan optimize:clear
```

Convenciones del repositorio: mensajes de commit cortos, en español, en una sola línea y sin firmas; el build compilado viaja siempre con los cambios; el trabajo se integra en `main`.

Checklist antes de desplegar: suite en verde · `npm run build` commiteado · migraciones revisadas y reversibles · permisos nuevos sincronizados en el servidor · variables de entorno nuevas creadas en producción.

— CIERRE

Un patrón repetido vale más que una solución brillante.

El sistema escala porque 88 pantallas comparten el mismo contrato. Antes de introducir una forma nueva de resolver algo, verifique si el patrón existente ya lo cubre: la consistencia es la característica más cara de mantener y la más fácil de perder.

XPLAIN
• PROCESS AI

XPLAIN SOLUTIONS S.A.C.

Tecnología, innovación e inteligencia artificial al servicio de su organización.

WEB

www.xplain.pe

CORREO

stefano@xplain.pe

DOCUMENTO

Manual del Programador · v2.0

